Self-Propelled Arm System

Design and Evaluation of a Low-Cost Autonomous Can Pickup Robot

---

Technical Project Report

Filip Chekalkin Huaijing Hou Spaska Janeva Kamen Kanev Giorgi Khundadze Xiaosheng Tan Stoyan Vlaev Weiye Yuan

Technical University of Munich, Campus Heilbronn, Germany

September 9, 2026

1 Abstract

This report presents a low-cost autonomous mobile manipulation system intended to collect lightweight litter in a controlled indoor environment. The envisioned robot follows a coarse search route, detects and collects objects, remembers additional candidates, returns to a marked disposal bin, and uses the bin marker to correct its pose before another cycle. The implemented prototype demonstrates the central can-pickup and bin-docking loop using learned can detection, fiducial-marker geometry, monocular relative-depth estimation, an explicit finite-state controller, approximate spatial memory, and empirically calibrated open-loop motion. It is not a general cleaning robot: the demonstrated object, bin, environment, and motion conditions are deliberately constrained. The report therefore separates the intended application from the achieved capability, explains the engineering decisions used to close the perception-action loop, and uses focused model checking to distinguish software guarantees from physical assumptions.

In eight consecutive trials using the latest parameter set and a preprogrammed square search path in an approximately 1.5×1.5 m elevator-lobby test area in one of the developer's apartment, six complete missions succeeded. One unsuccessful run knocked over the can and one failed to reacquire the bin marker.

Contents

2 Introduction and Project Objectives

Mobile manipulation combines navigation through a partially known environment with physical interaction. The selected task is deliberately concrete: a tracked vehicle must find a lightweight object, align with it, pick it up using a servo arm, return to a disposal bin, and release the object. Sensor measurements affect discrete decisions, decisions trigger continuous mechanical motion, and the changed physical state determines the next observation. This feedback chain makes the task a compact example of cyber-physical systems design [1].

2.1 Intended operating scenario

The motivating scenario is a room with a known static furniture layout after an event, such as the shared area of an apartment after a party. Lightweight litter may include cans, bottles, or paper objects. A suitably sized bin remains at a known landmark and carries a fiducial marker used both to find the bin and to relocalize the robot. A worker places the robot at a designated initial pose and selects a coarse route, for example a square or rectangular sweep. The robot searches along that route, yields control to visual tracking when an object is observed, bypasses unsuitable obstacles, collects one object, and records other candidates seen along the way. After returning to the bin, it deposits the object, corrects its pose from the marker, and begins the next cycle. Remembered candidates provide heuristic guidance rather than a guarantee that an object is still present.

Such a system could reduce repetitive collection work in a controlled space after deployment and calibration. This is an intended application, not a demonstrated labour-saving result. It assumes a static environment, appropriate objects and bin geometry, and sufficient calibration of the base response.

2.2 Prototype scope and contribution

The current plant does not realize the entire envisioned scenario. It reliably targets a selected beverage can as a representative item rather than arbitrary litter, uses a low custom bin, and depends on a controlled test environment. Coarse localization contains command-derived estimates and a small number of calibrated motion constants; it is therefore an engineering aid rather than a correctness-guaranteed navigation system. Obstacle extraction and bypass can be tested independently, but avoidance is disabled in the default integrated configuration.

Within this scope, the contribution is a working end-to-end demonstrator and a transparent account of the gap between the intended system and its realization. The prototype integrates:

- learned can detection suitable for onboard inference;
- AprilTag-based bin detection and pose estimation;
- relative-depth-based obstacle extraction;
- a finite-state mission controller with explicit success, retry, and failure paths;
- approximate spatial memory and command-based odometry; and
- configurable diagnostic, calibration, live-display, and recording tools.

Figure 1: Physical prototype and representative task objects: the JetTank platform, selected cans, the low marked bin, and an example non-target obstacle.

The remainder of this report introduces the plant and its operating assumptions, explains the control architecture, summarizes the implementation and development workflow, and identifies the observed capability boundary. Focused nuXmv models then show which controller properties follow structurally and which require additional physical contracts.

3 System Platform and Physical Constraints

3.1 Hardware platform

The physical plant is a Waveshare JETANK AI Kit B tracked mobile base equipped with a Jetson Nano, an IMX219-160 monocular camera, and a serial-servo arm with a gripper. The installed actuator set comprises four SCS15-AP servos and one SCS15-S servo [2]. Chassis motion is commanded through the JetBot `Robot` interface. Arm targets are sent through the FTServo/SCSCL interface as predefined joint poses. The camera is the only exteroceptive sensor used by the mission to observe the environment. Servo position, speed, load, voltage, temperature, moving state, and current can be read for diagnostics, but this telemetry is not used as closed-loop mission feedback.

Plant element	Implemented interface	Role
JetTank tracked base	JetBot differential motor commands	Translation and rotation of the mobile platform
Jetson Nano	Python and NVIDIA Jetson Inference	Onboard perception, decision making, HUD, and recording
IMX219-160 camera	320 by 240 RGB stream	Shared environmental input for can, Tag, and relative-depth processing
Serial-servo arm	Four SCS15-AP and one SCS15-S actuator	Camera repositioning, approach posture, grasp, carry, and bin insertion
Gripper	Configured open and closed poses	Mechanical capture and release of the selected can

The documented arm working radius is approximately 90–235 mm. The base uses three flat-top, unprotected 18650 cells in series: approximately 11.1 V nominal and 12.6 V when fully charged [2]. The available manufacturer information and physical inspection place the chassis envelope at roughly 230–240 mm long, 150–160 mm wide, and 110–120 mm high without the vertically extended arm; these ranges are descriptive rather than tolerance-controlled CAD dimensions.

Figure 2: JETANK AI Kit B component overview used to identify the principal chassis, camera, drive, arm, and electronics elements [2].

3.2 Consequences of the platform constraints

The base exposes no wheel-encoder, IMU, or Hall-based displacement and heading feedback to the application. Translation and rotation are therefore estimated from issued motor commands and elapsed time rather than measured directly. This loss of ego-motion observability is the main limitation on localization: pose error accumulates along open-loop routes, and an approximate coordinate cannot be treated as proof that the robot has reached a physical target. A fixed bin landmark and Tag-based relocalization can make a single return objective useful despite this limitation, but they do not turn the system into metric SLAM.

The arm similarly executes calibrated target poses rather than a Cartesian trajectory generated from a kinematic model. No force or pressure sensor confirms that the can is inside the gripper. Optional servo load and current telemetry remain diagnostic signals because they have not been characterized as reliable grasp evidence. Surface condition, battery state, payload, mechanical backlash, and object contact therefore influence repeatability.

The same monocular camera must serve three different consumers. Can detection requires appearance-based recognition; bin docking requires marker geometry; and obstacle avoidance requires a relative-depth field. The deployed 320 by 240 stream is a practical compromise for bounded onboard inference latency, heat, HUD rendering, and recording. It also limits long-range detail and makes geometric estimates less reliable near the image boundary. Monocular relative depth describes scene structure; unlike an IMU or wheel encoder, it does not directly measure the robot's own displacement or heading.

Subsystem	Available information	Important limitation
Tracked base	Direction, command speed, command duration	No direct displacement or heading feedback
Monocular camera	RGB frames at the configured resolution	No direct metric depth; blur and distortion affect geometry
Arm and gripper	Commanded poses and optional servo telemetry	No calibrated Cartesian end-effector state
Onboard computer	Real-time model inference and control	Finite throughput shared by inference, HUD, and recording

3.3 Physical task compatibility

The arm offers useful angular freedom and repeatable servo positioning, but the approximately 5.3 cm maximum gripper opening and approximately 4 cm minimum reachable claw height restrict the target set. The selected can is approximately 5 cm in diameter and 13.4 cm high. It satisfies these constraints and is used as a representative item; this does not imply generalization to wider bottles, paper, or arbitrary litter. The low custom bin is treated as a task fixture: it must allow arm clearance and keep the calibrated 4 cm Tag visible, but its external dimensions are not considered a controlled variable in this report.

4 System Requirements and Model

4.1 Operational design domain

The intended operating domain is a bounded indoor area with a static floor and furniture layout, no moving obstacles during a run, ordinary artificial lighting, lightweight target objects on the floor, and one fixed low bin carrying a visible Tag. The robot is manually deployed at a designated initial pose. Its main experiments have been conducted within an approximately 3 m by 3 m area; this describes the test setting, not a guaranteed perception or navigation range. Objects may be remembered only while the environment remains unchanged.

Figure 3: Representative indoor deployment showing the robot, target can, larger non-target object, and marked bin within a bounded test area.

4.2 Functional requirements and current coverage

For one requested pickup, the controller should initialize its components, execute a low-priority search routine, detect a can, visually align and approach, execute a grasp sequence, navigate coarsely toward the bin, reacquire its Tag, complete visual side docking, release the object, update its pose from the fixed landmark, and either terminate or plan another cycle. Visual acquisition must override search or map guidance. Arrival at an approximate coordinate is never treated as visual confirmation of a target.

```
SEARCH -> DETECT -> APPROACH -> PICKUP -> MAP-TO-BIN -> TAG-DOCK -> RELEASE -> RELOCALIZE ->
NEXT CYCLE
```

Intended capability	Current mechanism	Status or limitation
Search a known room	Predefined rotation or square-like routines	Implemented as coarse open-loop paths
Collect varied lightweight litter	DetectNet plus fixed arm sequence	Demonstrated for the selected can only
Reuse sightings	Vague Map candidate coordinates	Implemented as heuristic guidance
Return and deposit	Known bin pose, AprilTag/PnP, side docking	Demonstrated in the controlled setup
Correct the next-cycle pose	Tag-derived docking relocalization	Implemented conditionally on calibration and geometry
Avoid unsuitable objects	Depth-contour segmented bypass	Independent experimental capability; disabled by default in integration

The mission is modelled as a hybrid process. The finite-state controller provides a discrete state $q \in Q$, while the physical plant evolves continuously during each motor command. A transition is enabled by an event derived from perception, elapsed time, mission memory, or completion of a physical action. Docking has two roles: it provides the physical opportunity to deposit the object, and an accepted final Tag observation provides the opportunity to reset the robot pose relative to the fixed bin landmark.

4.3 Empirical motion model

Without wheel odometry, commanded motion is approximated by

$$\Delta s \approx k_{v(v,d)} \cdot v \cdot \Delta t, \quad \Delta\theta \approx k_{\omega(v,d)} \cdot v \cdot \Delta t,$$

where v is the requested command magnitude, d is direction, and the response factors k_v and k_ω are obtained from physical tests. Left and right turns are represented separately because equal motor commands do not necessarily produce equal angular motion. The model is used for coarse navigation and restoration of theoretical displacement; it is not presented as a full dynamic model of track–surface interaction.

For the report's $+y$-forward, $+x$-right convention, one calibrated segment updates the command-derived pose as

$$x_{k+1} = x_k + \Delta s_k \sin\theta_k, \quad y_{k+1} = y_k + \Delta s_k \cos\theta_k, \quad \theta_{k+1} = \theta_k + \Delta\theta_k.$$

Summing these increments gives the estimated pose after a command sequence. If an avoidance manoeuvre ends at p_A while the interrupted routine expected p_R, the coarse recovery vector is $c = p_R - p_A$. Its robot-relative bearing is estimated by $\alpha = \operatorname{atan2}(c_x, c_y) - \theta_A$ and its length by $\sqrt{c_x^2 + c_y^2}$. The controller can therefore turn through α and issue a calibrated forward segment for this estimated length. Because both p_A and the response factors are command-derived, this restores theoretical displacement rather than guaranteeing return to the physical path.

The map frame is fixed at mission initialization. Positive y points along the robot's initial forward direction and positive x points to its right. Positive heading rotates clockwise from $+y$ toward $+x$. This convention is also used when converting a visual horizontal error into a coarse target bearing.

4.4 Requirements traceability

ID	Requirement	Primary evidence
FR-1	Detect and approach a can	Detector HUD and approach recording
FR-2	Grasp and carry the selected can	Arm sequence test and full demonstration
FR-3	Locate and dock with the marked bin	AprilTag/PnP docking recording
FR-4	Release the can and complete or repeat	FSM log and end-to-end trial
FR-5	Use a predefined search when no stronger direction is available	Routine test and FSM trace
FR-6	Record candidate cans and reuse them after docking	Vague Map HUD and next-cycle trace
FR-7	Detect and bypass relevant obstacles	Static extraction and moving avoidance trials

5 System Design and Control Architecture

5.1 Runtime decomposition

The runtime separates perception, navigation, actuation, mission memory, and orchestration. `CanDetector`, `AprilTagBinDetector`, and `DepthSensor` translate camera frames into typed observations. Navigation components interpret these observations and return incremental motion decisions. `BaseController` and `ArmController` isolate hardware calls. `DemoStateMachine` owns the mission state and is the only component responsible for high-level transitions.

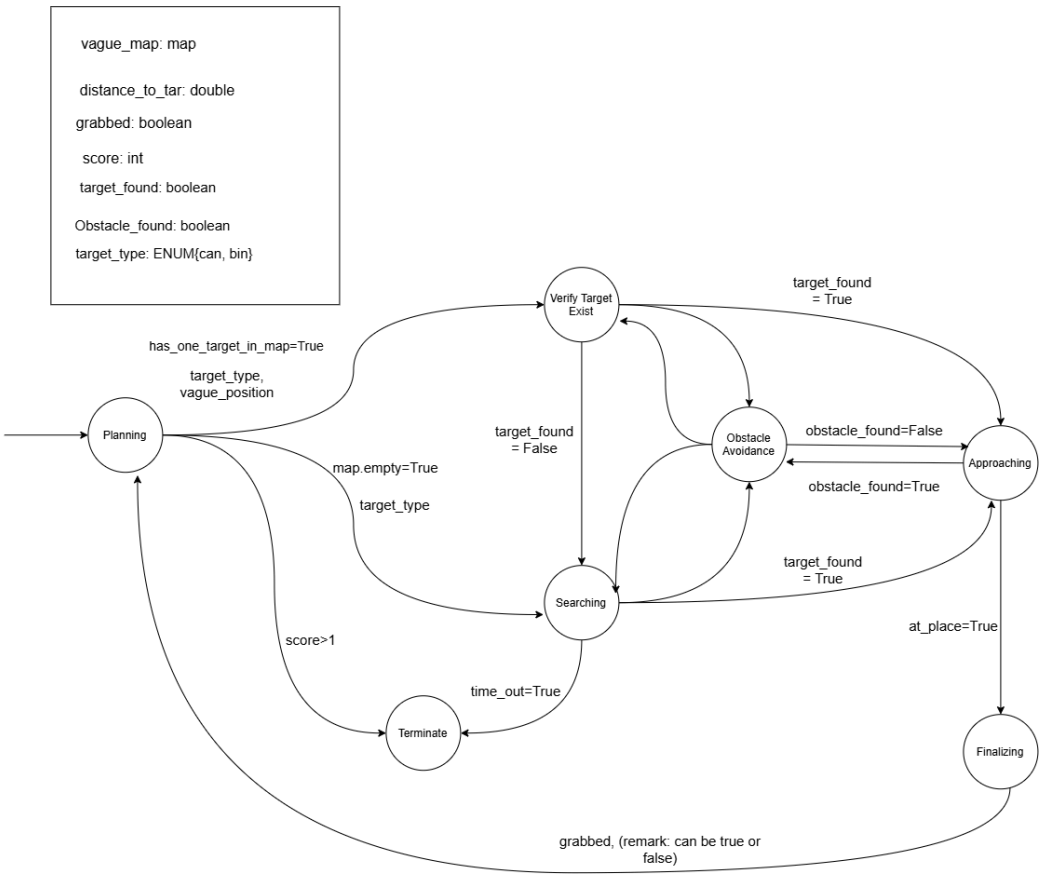

Figure 4: Early abstract mission model. The production implementation later introduced explicit map-navigation, alignment, side-docking, retry, and intermediate states.

5.2 Finite-state mission controller

The controller uses explicit `(state, event) -> next state` transitions. Planning selects either a remembered map target or a search routine. Once a visual target is detected, control passes through alignment, approach, and final multi-frame verification. A successful can verification triggers the grasp sequence; a successful bin verification triggers side docking and finalization. Obstacle avoidance may interrupt searching or approach, after which the controller either verifies the visual target or resumes the interrupted routine.

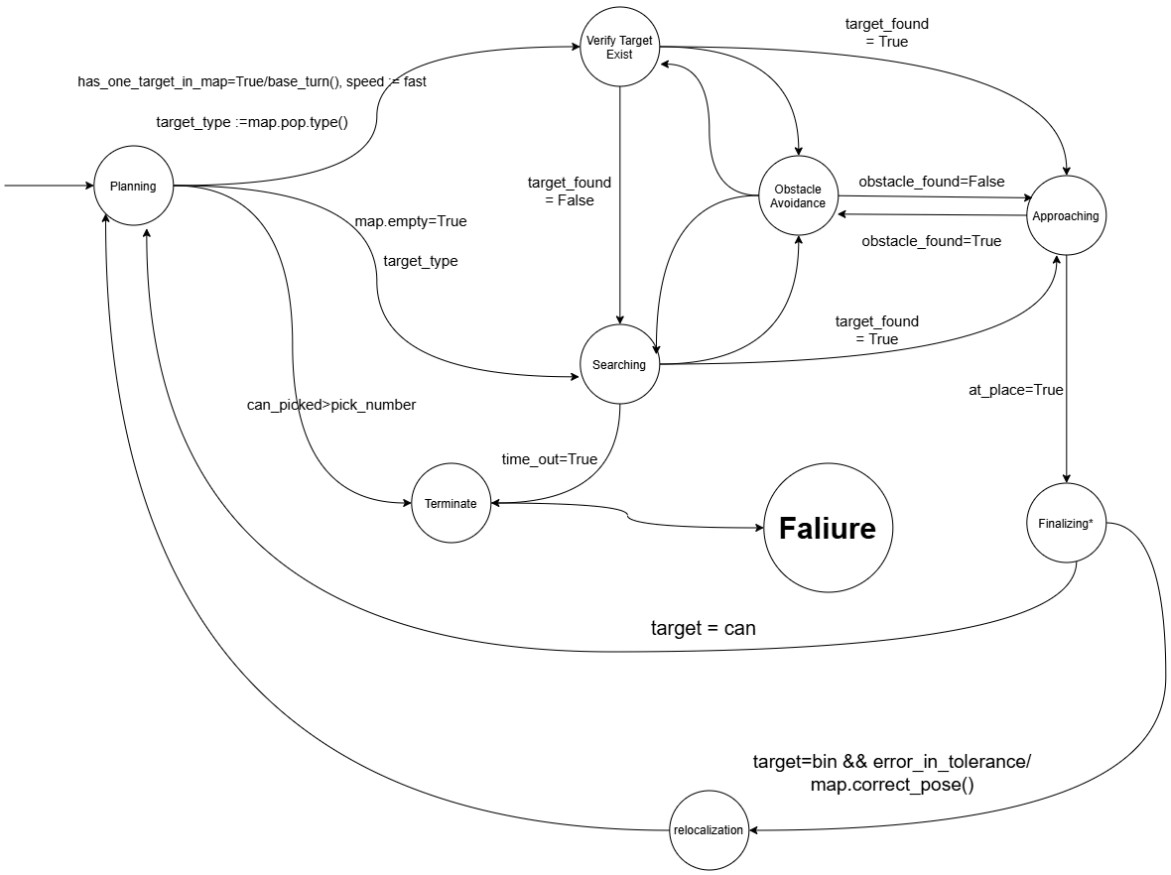

Figure 5: Simplified mission FSM showing planning, search, approach, avoidance, finalization, failure, and bin-based relocalization. It presents the control semantics rather than every production substate. Note that the syntax is semi-formal with many transitions omitted.

Figures Figure 4 and Figure 5 are retained because they document the evolution of the design. In the current implementation, `VERIFY_TARGET`, `MAP_NAVIGATING`, `ALIGNING`, `BIN_SIDE_DOCKING`, `INTERMEDIATE`, `DONE`, and `FAILED` make previously implicit behaviours explicit. This separation improves traceability: a target coordinate, a target observation, and a physically completed action are not treated as interchangeable facts.

5.3 Perception and decision making

The can path uses an SSD-MobileNet V1 ONNX model through the NVIDIA Jetson Inference DetectNet API to return bounding boxes and confidence values. The horizontal normalized error drives left/right alignment, while normalized bounding-box height provides the configured visual stopping condition before the fixed push-and-grasp sequence. Qwen-VL and YOLO-style tools assisted dataset preparation, but neither is the deployed runtime detector; referring to the running model simply as “YOLO” would therefore be inaccurate [3].

The bin path uses `pupil-apriltags` with the `tag36h11` family to detect a known marker. The configured black marker edge is 0.04 m. OpenCV loads the camera calibration, applies `solvePnP()` to the four detected corners and known marker size, and converts the resulting rotation with `Rodrigues()`; an OpenCV ArUco/AprilTag implementation remains a fallback detector [4]. The front approach stops according to the estimated PnP translation z, and side docking corrects orientation from the Tag-plane yaw derived from the PnP rotation. Lateral centring still uses the normalized horizontal displacement `error_x` between the image centre and the centre of the four-corner bounding box. This is retained as a legacy control variable and should eventually be replaced by the calibrated PnP translation `pose.x`, so that all three docking axes share one pose representation.

The obstacle path applies the pretrained Jetson Inference `fcn-mobilenet` DepthNet to obtain a relative-depth field; the model was not fine-tuned for the room. A foreground threshold is derived from estimated background depth, connected image regions are extracted, and a central collision corridor determines whether an object blocks the intended path. DepthNet values are relative model outputs, not millimetres or direct metric range. Shadows, reflections, and weak wall-floor contrast can therefore change the extracted contour even when the physical layout is unchanged.

5.4 Camera geometry and frame spaces

The current implementation distinguishes raw camera coordinates from rectified coordinates. Depth inference and obstacle extraction use calibrated rectification by default. Can inference remains in raw image space by default

because its training images were not rectified. AprilTag pose estimation uses raw corner coordinates together with the camera matrix and distortion coefficients. Explicit frame-space conversion prevents a bounding box measured in one image space from being sampled at the wrong point in another.

Engineering rationale. A globally rectified stream would appear simpler, but it would add resampling to consumers whose training data are raw. The implemented design instead makes each consumer's coordinate space explicit.

5.5 Search routines, Vague Map, and coarse navigation

When no visual target or remembered candidate has higher priority, the robot executes a predefined routine. Available routines include continuous rotation and square-like sweeps assembled from calibrated drive and turn segments. An extended rotation is available after coarse arrival near the bin when the Tag has not yet been reacquired. Vision is evaluated during routine execution and immediately overrides this lowest-priority motion.

The `VagueMap` is an approximate spatial memory rather than SLAM or a correctness-bearing localization subsystem. `CommandOdometry` updates robot pose from calibrated motor commands. Additional can observations may be stored as coarse candidates, and a known bin pose guides the platform home after pickup. On arrival, vision must reacquire the Tag. Following an accepted docking localization update, the robot pose and remembered can coordinates are transformed consistently so that the next cycle can use corrected heuristic directions. The map cannot repair an inaccurate original can observation or compensate for an object moved by a person.

5.6 Pickup and side docking

The pickup sequence is intentionally divided into visually guided and open-loop phases. Vision aligns the base and selects the stopping point before the arm descends. Once the claw obstructs the camera, the push, close, lift, and carry actions follow calibrated durations and servo poses.

Bin placement similarly combines calibrated pose geometry with repeatable mechanical actions. The robot approaches until PnP z reaches the configured standoff, rotates the arm camera to a fixed side-view pose, performs a calibrated coarse base turn, and waits for the camera view to settle. It then corrects lateral image displacement using the legacy `error_x` term and corrects orientation using PnP-derived yaw. Once both active conditions are stable, the base remains stopped while the arm moves through insertion, release, and home poses.

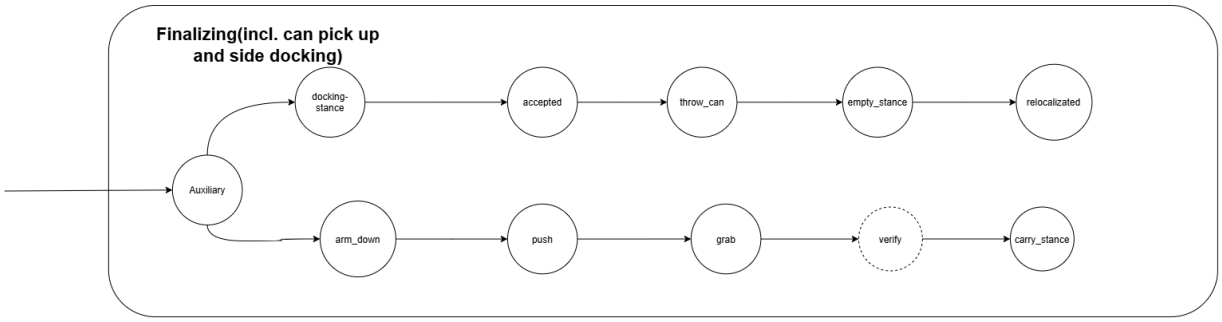

Figure 6: Conceptual finalization branches for can pickup and bin placement. The labels summarize physical stages; the implementation realizes bin placement through calibrated side docking, insertion, release, safe-home motion, and relocalization.

Figure 7: Low custom bin with the 4 cm fiducial marker and two deposited cans after demonstration runs.

5.7 Reactive obstacle avoidance

The implemented avoidance behaviour is best described as depth-contour-based reactive segmented bypass. The robot identifies a blocking contour, chooses a side from local clearance, turns until the locked obstacle leaves the collision corridor, executes offset and passing segments, and records theoretical displacement needed to return toward the interrupted route. This behaviour is available in focused tests, while the default integrated configuration keeps the avoidance strategy disabled because perception robustness, not path computation, remained the limiting factor.

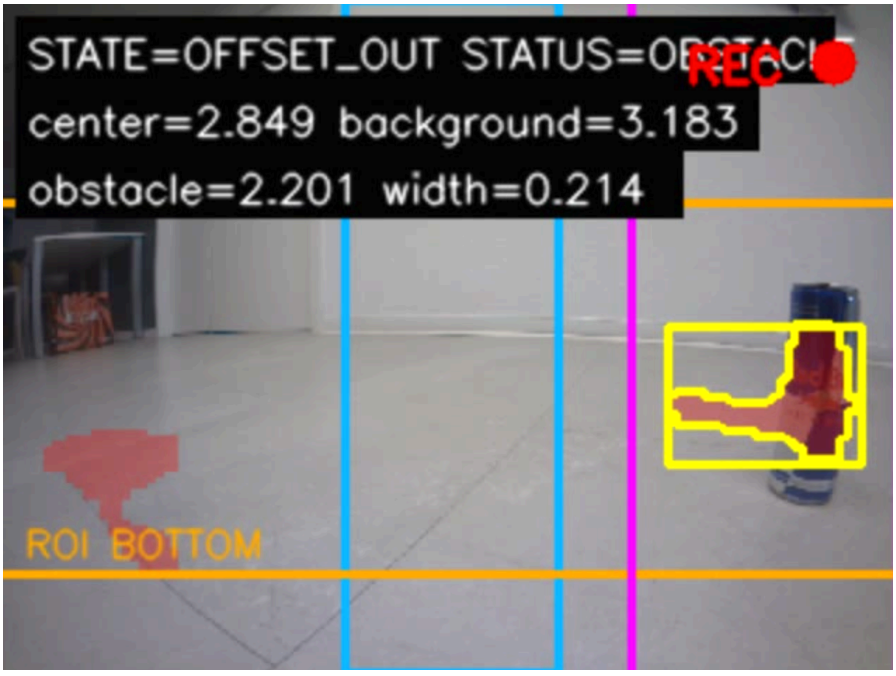

Figure 8: Recorded obstacle-debug HUD showing the collision corridor, extracted obstacle contour, selected side, tangent boundary, and relative-depth statistics.

The use of an obstacle edge as a local steering cue is related to the range-based local-tangent idea of TangentBug [5]. However, the present controller implements only the local tangent intuition. It does not construct the complete local tangent graph, implement the original boundary-following leave condition, or inherit its convergence result. The citation is related work rather than a claim of algorithmic equivalence.

6 Implementation and Development

6.1 Software organization

The Python codebase groups runtime responsibilities under `demo_core`. Project configuration selects runtime features, while empirical parameters hold camera settings, motion response samples, thresholds, and arm poses. Deprecated experimental keys are rejected rather than silently accepted. Hardware-specific imports are delayed until real components are started, allowing most state, geometry, and navigation behaviour to be tested on a development computer. The principal executable is complemented by focused Notebook interfaces for calibration and real-hardware observation.

Module group	Responsibility
`state_machine`, `fsm_types`	Mission state, events, transitions, retries, and context
`perception`, `depth_vision`, `camera_geometry`	Visual inference, PnP, depth sampling, and coordinate spaces
`navigation`, `turn_response`	Incremental visual approach and calibrated docking corrections
`vague_map`, `searching_routine`	Spatial memory, command odometry, and low-priority search
`robot_control`	JetBot base commands and serial-servo pose execution
`tangentbug`	Depth-contour analysis and local bypass candidates

6.2 External software and development toolchain

External software is used throughout the project lifecycle, but not every tool runs onboard the robot. The distinction between development assistants, deployed runtime dependencies, and offline verification tools prevents dataset tooling from being mistaken for runtime perception and prevents verification of an abstract controller from being mistaken for proof of physical performance.

Software	Lifecycle	Role in the project	Status
Qwen3-VL-235B-Instruct	Dataset preparation	Reviews images and proposes candidate annotations; it is not called by the running robot.	Development assistant
YOLO tools	Dataset preparation	Produces initial pseudo-labels that are subsequently checked and corrected.	Development assistant
NVIDIA Jetson Inference	Onboard runtime	Runs the SSD-MobileNet V1 DetectNet model and pretrained `fcn-mobilenet` DepthNet.	Deployed
`pupil-apriltags`	Onboard runtime	Primary `tag36h11` detector for the known bin marker.	Deployed
OpenCV	Runtime and development	Provides calibration, PnP, coordinate conversion, fallback Tag detection, contours, and HUD rendering.	Deployed
JetBot	Onboard runtime	Provides the tracked-base motor command interface.	Deployed
FTServo / SCSCL	Onboard runtime	Provides serial-servo commands and optional actuator telemetry.	Deployed
Jupyter / ipywidgets	Calibration and evaluation	Provides focused tests, live displays, parameter controls, and recording.	Development tool
nuXmv	Offline verification	Checks an abstract representation of the finite-state mission controller.	Verification tool

The project-specific contribution lies in selecting, adapting, and integrating these assets into one coherent perception-action pipeline. Qwen3-VL-235B-Instruct and YOLO are used before deployment rather than during a mission. DetectNet, DepthNet, marker detection, OpenCV geometry, JetBot control, and the servo interface participate directly in the running system. nuXmv operates offline on the controller abstraction [6]. The DetectNet model path and labels are configurable, allowing compatible model assets to be exchanged without changing the mission controller; this is not a generic runtime switch among unrelated inference frameworks.

6.3 Dataset and model workflow

Can images were collected from the onboard viewpoint over different target positions, apparent sizes, and backgrounds. Qwen3-VL-235B-Instruct-assisted review and YOLO-based pseudo-label generation accelerated annotation, after which candidate labels were inspected and corrected. The frozen dataset contained 671 training images and 194 validation images, including retained empty and hard-negative frames. Training and conversion were performed on a desktop system with an RTX 5070, while the deployed artifact is a 300 by 300 SSD-MobileNet V1 ONNX model consumed through the standard DetectNet API on the Jetson Nano. This separation avoids mistaking a dataset assistant or desktop training framework for the onboard detector.

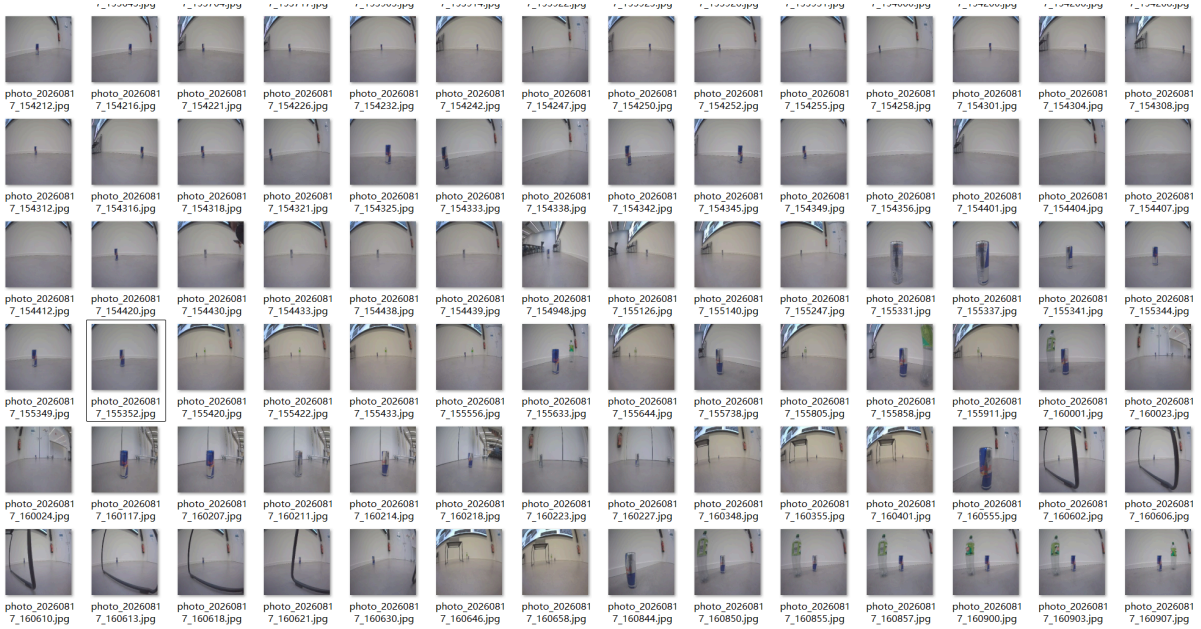

Figure 9: Overview of onboard can-image samples spanning different apparent sizes, positions, and backgrounds.

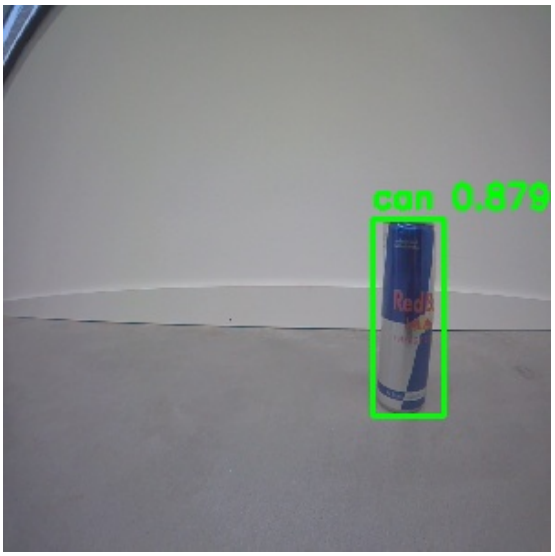

Figure 10: Example output from the deployed can detector.

The selected native DetectNet model achieved a best observed validation mAP of 0.664. On frozen robot-domain sets, its F1 score was 0.839 for photographs and 0.924 for video frames. These values summarize the controlled evaluation sets and are not presented as general object-detection performance. Detailed epoch schedules, resource traces, and per-threshold counts are retained in the training record but are not required to explain the system architecture.

6.4 Calibration and diagnostic tooling

The development workflow uses focused tools instead of tuning every behaviour through the full mission. The central JSON parameters include direction-specific base response samples, camera dimensions, detector assets, approach thresholds, Tag geometry, docking tolerances, and arm poses. The 320 by 240 calibration file is consumed where metric camera geometry is required. Depth processing uses rectified frames, can inference remains in the raw training-image space, and AprilTag PnP uses raw corners together with the calibrated camera matrix and distortion coefficients.

Separate interfaces measure base speed-time response, arm poses and telemetry, camera intrinsics, AprilTag PnP stability, can stopping distance, dual-object depth separation, side docking, and full integration. Live HUDs expose state, bounding boxes, Tag geometry, and relative-depth statistics. CSV logs and optional recordings preserve evidence without requiring verbose Notebook output.

7 Experimental Observations

Eight consecutive full-mission trials used the latest parameter set, a preprogrammed square search routine, and an approximately 1.5×1.5 m square area in the elevator lobby of XX Building. Success required autonomous can detection, pickup, return to the marked bin, docking, release, and entry into the configured terminal state. Six runs succeeded; one run knocked over the can and one failed to find the bin marker.

This small, single-site batch demonstrates that the complete pipeline can operate repeatedly, but it is not sufficient to estimate a general success probability. The trials used one scene, one parameter set, and no randomized placement protocol; subsystem outcomes were not counted separately, and no confidence interval or cross-environment comparison is reported.

Evaluation	Trials	Completed	Observed failures
Complete mission	8	6 (75%)	Can knocked over: 1; bin marker not found: 1

Figure 11: Representative before-and-after views of one collection cycle. The pair illustrates task completion but is not itself the basis of the success-rate calculation.

7.1 Observed operating bounds and failure modes

The following statements are appropriate as observations pending quantitative consolidation:

- equal numerical motor commands produce different physical left and right rotations;
- open-loop distance and heading drift accumulate during long routes;
- the 320 by 240 stream limits fine detail and makes distant or edge-of-frame geometry less stable;
- PnP yaw becomes sensitive when the tag is far from the image centre or observed at an unfavourable angle;
- movement of the arm-mounted camera temporarily invalidates cached tag geometry;
- the claw obscures the target during the final pickup motion, requiring an open-loop completion sequence; and
- shadows, reflections, and weak floor-wall contrast can destabilize relative-depth obstacle contours;
- the gripper geometry restricts the demonstrated target and bin dimensions; and
- simultaneous live display, inference, and recording compete for limited compute and I/O capacity.

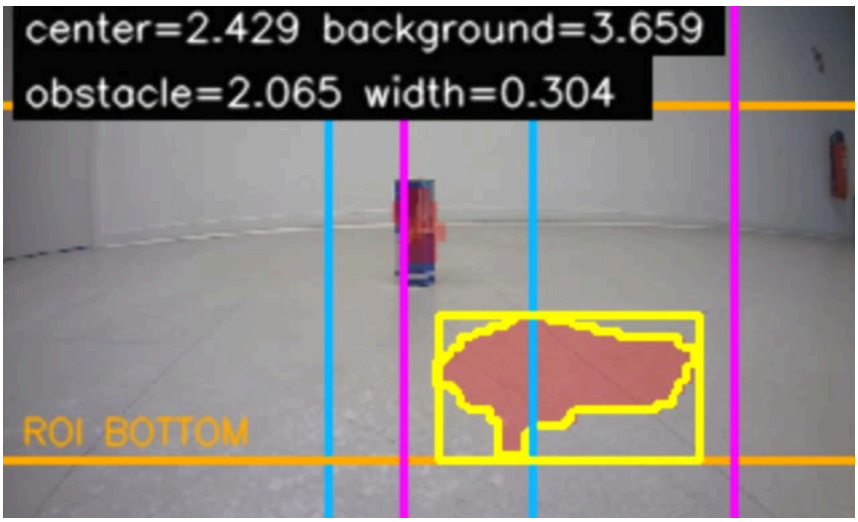

Figure 12: Example failure of relative-depth contour recovery: scene appearance and shadow merge the foreground response into an incorrect obstacle shape.

These are not mathematical invariants. Their report value lies in explaining why the controller uses direction-specific calibration, camera-settle intervals, multi-frame confirmation, staged docking, and narrowly scoped test

tools. The avoidance implementation was not promoted to a default integrated capability because the relative-depth observation was less robust than the segmented bypass logic itself.

8 Preliminary Formal Verification

The intended cleaning cycle provides a useful specification, but the present plant cannot justify every physical premise needed by that specification. Formal verification was therefore applied to three focused finite-state reference models rather than to a monolithic claim that the room will always be cleaned, as we don't assume we have the engineering maturity to develop a completely correct controller for such a challenging aim. The models were derived from the current implementation of our submitted codebase and cover (1) Tag-based relocalization and the next mission cycle, (2) the deterministic can-pickup sequence, and (3) side-docking pose control. Continuous quantities and physical outcomes are represented by finite predicates. This makes the boundary between software guarantees and assumptions about perception, mechanics, and the environment explicit.

8.1 Method and interpretation

The models were checked with nuXmv 2.1.0 / NuSMV 2.7.0 under (WSL2) Ubuntu 22.04. The automated runner enables cone-of-influence reduction, checks that each transition relation is total, and rejects any unexpected false property. Deliberately unstrengthened properties are named `EXPECTED_FALSE_*`; their counterexamples identify a missing physical or environmental contract. A strengthened property is then checked after that contract is introduced. The three SMV models, raw tool output, and detailed argument notes are retained as supplementary project artifacts rather than reproduced as source code in the report.

Reference model	True	Expected false	Unexpected false	Total relation
Relocalization and next cycle	10	4	0	Yes
Fixed can pickup	7	2	0	Yes
Physical side-docking pose	7	4	0	Yes

All 24 target or structural properties passed, while all 10 false results were intentional boundary tests. These counts do not measure physical success rate; they summarize model-checking outcomes within the stated abstractions.

8.2 Tag relocalization and the next cycle

The first model separates coordinate consistency from mission progress. A successful Tag correction always changes the software pose source to a Tag-derived pose. It bounds the physical pose only under `A_tag`, the contract that the accepted Tag pose, camera calibration, and camera mount have bounded error. Under the additional navigation-error contract `A_nav`, the corrected bin-relative coordinate remains valid weakly until the next docking event:

```
G ((TAG_FIX & A_tag & A_nav) -> X (bin_relative_correct W DOCKED))
```

Here weak-until allows the coordinate to remain valid even if docking never occurs. Correct use of a remembered can additionally requires `P1_can`: bounded map-observation error and no external relocation. A separate contract `A_progress` is needed for eventual docking. Thus coordinate consistency does not by itself imply liveness.

8.3 Deterministic can pickup

The pickup model follows the implemented command order:

```
CAN_VISUALLY_STABLE -> ARM_DOWN -> FIXED_PUSH -> CLOSE_GRIPPER -> CARRY_POSE -> PICKUP_COMPLETE
```

Ordering invariants establish that push follows arm-down, gripper closure follows push, and carry follows closure. However, `mark_pickup()` is software bookkeeping and is not physical evidence that the can is held. The unqualified implication from software completion to physical grasp therefore has an expected counterexample. Stable pickup follows only under the combined contracts for bounded pre-grasp can pose (`A_can_pose`), calibrated arm geometry (`A_arm_geometry`), sufficient fixed push displacement (`A_push`), gripper retention (`A_grip`), and successful actuator execution (`A_actuator`).

This result accurately describes the present implementation: vision establishes the pre-grasp pose, but after the arm descends and obscures the target, push and grasp completion are open-loop physical actions.

8.4 Side-docking pose control

The docking model captures the sequence from the front PnP-z stop through the calibrated side-entry turn and final visual correction. In this abstraction, `x_ok` denotes acceptance of the current image-centre `error_x` condition rather than a metrically estimated lateral translation:

```
FRONT_PNP_APPROACH -> FIXED_90_DEGREE_TURN -> CORRECT_X -> CORRECT_YAW -> DOCKED
```

The three components do not yet have equal status. Longitudinal distance `z_ok` is established before the side turn from PnP `pose.z`, and orientation `yaw_ok` is subsequently closed from the PnP rotation. The lateral condition `x_ok`

is closed using the legacy image-space `error_x`, not PnP `pose.x`. The final stability condition tests the lateral image error and PnP yaw, but it does not re-close z. Consequently, acceptance of the two side-view conditions does not by itself imply a bounded three-axis metric docking pose: an expected counterexample allows correction motions to consume the earlier z margin.

The complete result requires four contracts: PnP-z establishes an initial longitudinal margin (`A_pnp_z`); the calibrated 90-degree turn enters a valid side-view correction basin (`A_turn90`); image-centre/yaw correction converges (`A_xy_convergence`); and its accumulated longitudinal disturbance is smaller than the initial margin (`A_z_coupling`). Under these assumptions, the model reaches `DOCKED` with all three abstract acceptance predicates satisfied. Replacing `error_x` with calibrated PnP `pose.x` is future work needed before interpreting the lateral predicate directly as a metric pose bound.

8.5 Scope of the evidence

The verification proves conditional statements about controller structure. It does not independently establish camera accuracy, actuator repeatability, grasp force, metric pose error, or the probability of mission success. Those claims require calibration and repeated physical trials. The useful result is instead diagnostic: the counterexamples expose exactly which sensing or actuation contracts must be justified before a stronger physical conclusion can be made.

9 Discussion

The envisioned system is simple from a user's perspective: deploy the robot, provide a route and bin landmark, and allow repeated collection cycles. Realizing that behaviour exposes a recurring CPS trade-off. Architectural clarity can manage hardware uncertainty, but it cannot remove uncertainty from the physical plant. An explicit FSM makes actions and recovery paths understandable, parameterized controllers make calibration reproducible, and Vague Map memory improves direction selection. None of these mechanisms turns open-loop tracks into a metrically accurate mobile base.

The most successful behaviours use a hierarchy of information. A coarse command model supplies approximate direction, object detection takes control once a target is visible, and marker geometry supplies the final local correction. Fixed mechanical poses are used when vision is occluded or when a repeatable movement is more reliable than noisy geometry. This hierarchy is more appropriate for the prototype than pretending that every subsystem offers the same precision.

Several engineering workarounds are therefore defensible within the demonstrated environment: separate left/right rotation tables, a calibrated fixed side turn before Tag refinement, PnP z for bin approach, and a fixed push distance after can alignment. These are limited and visible substitutions for missing feedback rather than hidden claims of generality. Performance can change with surface, battery, payload, camera mounting, and marker geometry, and the system does not estimate all of these factors online.

The focused model-checking results sharpen this discussion. They show that the controller preserves the intended command order and can compose local guarantees, while also demonstrating why software completion flags cannot replace physical evidence. In particular, bounded map reuse depends on observation and navigation contracts, physical pickup depends on a gripper-retention contract, and three-axis docking depends on bounding the longitudinal disturbance introduced by lateral and yaw corrections.

The gap between the intended scenario and the current demonstrator is therefore specific rather than absolute. The system automates the central can-to-bin cycle and exposes the state, geometry, and parameters needed to tune it. It does not yet establish general object compatibility, robust depth-based avoidance across ordinary lighting, or drift-bounded room-scale navigation. Those limitations identify where additional sensing would remove assumptions rather than merely add software complexity.

10 Conclusion and Future Work

This project integrates onboard visual recognition, fiducial pose estimation, monocular relative depth, explicit mission control, approximate spatial memory, and a servo arm into a recorded autonomous can-pickup and bin-docking demonstration. The achieved system can execute the central perception-action sequence in a controlled setup and can use a fixed bin Tag to support coarse next-cycle relocalization. Its main result is not a general cleaning or navigation algorithm, but a functioning and inspectable pipeline adapted to inexpensive hardware.

The remaining gap is concentrated in generality and physical evidence. The present gripper and fixed sequence do not support arbitrary litter; relative-depth obstacle extraction is not robust enough for default integration; and command odometry does not bound long-route drift. The state-based design and three focused nuXmv models

make these limits explicit by distinguishing unconditional command-order guarantees from conclusions that require perception, navigation, gripper-retention, and actuator contracts.

Future work should first improve observability of the mobile base. Wheel encoders and an IMU would enable closed-loop distance and heading control and substantially reduce Vague Map drift. A calibrated depth camera could replace relative-depth thresholds with more stable scene geometry. Further improvements include camera-to-arm extrinsic calibration, a kinematic arm model, a wider or lower-reaching end effector, force or current-based grasp evidence after controlled characterization, and repeated trials across surfaces, battery levels, lighting, and object placements. Each upgrade can be reflected in the verification model by replacing a strong environmental assumption with a measured or closed-loop system guarantee.

References

[1] E. A. Lee, "Cyber Physical Systems: Design Challenges," in *11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing*, 2008, pp. 363–369. doi: 10.1109/ISORC.2008.25.

[2] Waveshare, "JETANK AI Kit." [Online]. Available: https://www.waveshare.com/wiki/JETANK_AI_Kit

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91.

[4] E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 3400–3407. doi: 10.1109/ICRA.2011.5979561.

[5] I. Kamon, E. Rimon, and E. Rivlin, "TangentBug: A Range-Sensor-Based Navigation Algorithm," *The International Journal of Robotics Research*, vol. 17, no. 9, pp. 934–953, 1998, doi: 10.1177/027836499801700903.

[6] R. Cavada *et al.*, "nuXmv: a new state-of-the-art symbolic model checker," in *Computer Aided Verification (CAV 2014)*, in Lecture Notes in Computer Science, vol. 8559. Springer, 2014, pp. 336–342. doi: 10.1007/978-3-319-08867-9_22.